GOOGLE ANTIGRAVITY 2.0 ECOSYSTEM

안티그래비티 환경 종합 아키텍처 보고서

[제1권] 플랫폼 생태계, 4대 서피스, 비동기 코어 런타임 및 보안 거버넌스 체계

PLATFORM SURFACES	AI FOUNDATION CORE	TARGET PROJECT	PUBLICATION DATE	ARCHITECT
Antigravity 2.0 / IDE / CLI / SDK	Gemini 3.8 Flash & Pro	FC (Unreal Engine 5.8 C++)	2026-10-22	Lead Systems & AI Architect

1. 개요 및 플랫폼 비전 (Executive Vision)

Google Antigravity(안티그래비티)는 대규모 코드베이스의 설계, 리팩토링, 디버깅 및 자율 검증을 위해 설계된 **AI-First 에이전틱 개발 플랫폼(Agentic Development Platform)**이다. 단순한 코드 자동완성(Copilot형) 보조 도구를 넘어, 개발자와 동등한 권한과 도구를 가지고 프로젝트 환경을 능동적으로 탐색하고 실행 루프를 수행하는 **자율 페어 프로그래머(Autonomous Pair Programmer)** 모델을 구현한다.

1. 멀티 서피스 단일성
데스크톱 2.0, VS Code 기반 IDE, 헤드리스 CLI(agy), Python SDK가 동일한 에이전트 코어와 도구셋을 공유.

2. 리액티브 이벤트 기상
무한 루프 폴링을 원천 차단하고, 서브에이전트 메시지, 백그라운드 태스크 완료 시 자동 기상(Wakeup)하는 이벤트 구동 모델.

3. 계층형 서브에이전트
컨텍스트 윈도우 오염 방지를 위해 Research, Self, Custom 에이전트를 생성하고 branch/share 워크스페이스 격리 지원.

4. 샌드박스 보안 거버넌스
4단계 도구 실행 정책, 파일/네트워크 접근 제어, 터미널 샌드박스를 통해 시스템 파괴를 원천 방지하는 안전망 제공.

2. 플랫폼 최상위 토폴로지 (System Topography)

[Google Antigravity End-to-End Architecture]

Desktop App (2.0)	Antigravity IDE	CLI (agy)	Python SDK
Electron / Sidebars / Panes	VS Code Engine / Tab / Ctrl+I	Terminal TUI / CI/CD Automation	Async Orchestration / Pipeline

↓ **Unified Agentic Core Runtime** (Gemini 3.8 Multi-Modal CoT / Tool Calling / Streaming Bus) ↓

Context & Memory	Execution Engine	Subagents & IPC	Security & Sandbox
Progressive Disclosure / Rules / Skills / Transcripts	Reactive Wakeup / Background Tasks / Cron	Dynamic Types / Branch & Share Workspaces	Tool Policies / Terminal Jail / Allow/Deny Rules

3. 보고서 구성 안내 (Report Structure)

본 보고서는 총 3권으로 구성되며, 각 권은 안티그래비티 환경의 상이한 아키텍처 계층을 심층 분석한다.

권차	주제 (Title)	주요 분석 대상	문서 형태
제1권 (본서)	플랫폼 생태계 & 코어 런타임	4대 서피스, 비동기 추론 루프, 서브에이전트 IPC, 스케줄링 및 보안 샌드박스	코어 런타임
제2권	커스터마이징 & 확장성 프레임워크	Rules, Skills, Plugins, Hooks, MCP 프로토콜, 아티팩트 및 트랜스크립트 상태 관리	확장 규격
제3권	FC 언리얼 5.8 실전 엔지니어링 & 루프	4계층 네트워크 권한, 6대 도메인 룰셋, pipeline.py 계층 러너, Ground Truth 에셋 추출	실전 구현

1. 안티그래비티 4대 서피스 (Interface Surfaces) 아키텍처

안티그래비티는 단일 사용 형태에 얽매이지 않고, 개발자의 작업 맥락(Context)에 따라 최적화된 4가지 독립 접점을 제공한다. 모든 서피스는 백엔드에서 **동일한 에이전트 바이너리와 설정 저장소**를 공유하므로 상태 연속성이 보장된다.

1. Antigravity 2.0 (Desktop Electron App)

통합 대시보드

IDE와 독립적으로 실행되는 풀스택 데스크톱 클라이언트. 에이전트의 전체 오케스트레이션과 백그라운드 태스크 모니터링에 특화.

- **Left Sidebar:** New Conversation, Projects Switcher, Scheduled Tasks(Cron/Timer), Skills/Plugins 매니저.
- **Chat Canvas:** 슬래시 커맨드(/), 컨텍스트 멘션(@: 파일, 규칙, 세션, MCP), 미디어 드래그 앤 드롭 지원.
- **Auxiliary Pane (보조 패널):** Subagents 실시간 상태, Background Tasks, Artifacts(문서/다이어그램), Terminals, Files Changed 시각화.

2. Antigravity IDE (VS Code Fork)

코드 작성 특화

개발 환경 내부에 네이티브로 통합된 AI-First 에디터. 3단계 AI 모달리티를 통해 타이핑 단위부터 대규모 리팩토링까지 지원.

- **Passive (Tab):** 단일 키로 다음 의도 예측(Next-Intent). Autocomplete, Supercomplete, Tab to Jump, Tab to Import.
- **Instructive (Ctrl+I):** 에디터 내부 선택 영역 인라인 명령. 집중된 리팩토링, 주석/Docstring 생성, 보일러플레이트 작성.
- **Collaborative (Sidebar Agent):** 파일 생성/편집, 빌드 실행, MCP 도구 연동을 포함하는 대규모 페어 프로그래밍.

3. Antigravity CLI (agy)

헤드리스 & 터미널

터미널 환경에서 빠르고 경량화된 에이전트 상호작용 및 CI/CD 자동화 파이프라인 연동을 위한 전용 TUI 도구.

- **Launch & Auth:** agy 명령어로 즉시 구동, OAuth/API 키 기반 간편 인증.
- **Interactive TUI:** 터미널 내에서 슬래시 커맨드 및 대화형 인터페이스 제공 (Ctrl+D로 종료).
- **Settings:** ~/.gemini/antigravity-cli/settings.json을 통한 모델 및 터미널 렌더링 세부 튜닝.

4. Antigravity Python SDK (google-antigravity)

파이프라인 프로그래밍

파이썬 스크립트, 자동화 테스트 및 CI/CD 워크플로 내에서 프로그래밍 방식으로 에이전트를 스폰하고 오케스트레이션.

- **Async Context Manager:** async with Agent(config) as agent:를 통한 완전한 생명주기 관리.
- **Event Streaming:** 응답 토큰뿐 아니라 내부 사고(response.thoughts) 및 도구 호출(response.tool_calls) 실시간 가로채기.
- **Tool Wiring & Policies:** 커스텀 도구 바인딩 및 Capability 정책의 코드 레벨 제어.

2. 4대 인터페이스 서피스 상세 비교 분석표

구분 항목	Antigravity 2.0 (Desktop)	Antigravity IDE	CLI (agy)	Python SDK
주요 사용 목적	전체 프로젝트 조망, 태스크 스케줄링, 서브에이전트 총괄	실시간 인라인 코딩, 자동완성, 심볼 렌즈, 코드 리팩토링	터미널 작업 중 빠른 질의, 서버 환경, 경량 스크립팅	자동화 파이프라인, 테스트 하네스, 커스텀 에이전트 빌드
UI 형태	Electron 독립 데스크톱 윈도우	VS Code 기반 통합 개발 환경	터미널 TUI (Rich Console)	Headless Python 라이브러리
컨텍스트 주입	@ 멘션 메뉴, 파일 드래그 앤 드롭	열린 탭, 커서 위치, 진단 목록 자동 수집	명령 인자, 파이프(stdin), 파일 경로	API 파라미터, 인메모리 버퍼 객체
백그라운드 제어	전용 Auxiliary Tasks 패널에서 직관적 제어	사이드바 패널 연동 및 상태 표시	CLI 백그라운드 프로세스 관리	Asyncio 이벤트 루프 기반 제어
커스터마이징 범위	전역 및 프로젝트 설정, GUI 관리기	.agents/ 및 인라인 코드 렌즈	settings.json 및 플래그	코드 기반 CapabilitiesConfig 주입

[Architectural Insight] 서피스 간 상태 일관성 보장 메커니즘 Unified Backend

모든 서피스는 로컬 파일 시스템의 ~/.gemini/config/ 및 프로젝트 루트의 .antigravity/ 또는 .agents/를 공유한다. 예를 들어 IDE에서 작성된 규칙이나 스킬은 데스크톱 2.0 및 CLI에서도 즉시 동일한 우선순위로 로드되며, 트랜스크립트 로그(transcript.jsonl) 역시 표준화된 경로에 기록되어 도구 간 세션 연속성을 완벽히 유지한다.

1. 에이전틱 코어 런타임 추론 루프 (Reasoning & Dispatch Loop)

안티그래비티 코어 런타임은 Gemini 3.8의 멀티모달 추론 역량을 기반으로 **관찰(Observe) → 분석/사고(Deliberate/Think) → 도구 호출(Dispatch) → 피드백 평가(Evaluate)**의 폐루프(Closed Loop) 아키텍처를 형성한다.

1. Context Ingestion
사용자 인텐트, 계층적 규칙(AGENTS.md), 점진적 스킬 메타데이터, 트랜스크립트 이력 병합

→

2. Chain-of-Thought
심층 추론(Deep Thinking), 변경 파급 효과 분석, 최소 수정 원칙 및 실행 전략 수립

→

3. Tool Call Dispatch
보안 정책 필터링 통과 후 run_command, edit_file, read_file 호출

→

4. Feedback Loop
컴파일러/명령어 표준 출력 및 종료 코드 수집, 오류 발생 시 자동 수정(Auto-Patch)

2. 실시간 스트리밍 버스 (Real-Time Streaming Pipeline)

런타임은 에이전트의 내부 상태 변화를 실시간 스트림(Stream) 이벤트로 외부에 전파하여, UI 레이어와 SDK 클라이언트가 지연 없이 에이전트의 사고와 행동을 시각화할 수 있도록 지원한다.

1. Thought Stream

```
async for thought in resp.thoughts:
  render_thinking_bubble(thought)
```

모델의 CoT(Chain-of-Thought) 추론 조각을 실시간 수신하여 에이전트의 문제 해결 방향성을 투명하게 공개.

2. Tool Call Stream

```
async for call in resp.tool_calls:
  audit_and_dispatch(call.name, call.args)
```

강타입화된 도구 호출 객체를 실시간 전송. 보안 가드레일이 실행 전 인수를 사전 가로채기(Interception) 가능.

3. Content Token Stream

```
async for token in resp:
  stdout.write(token)
```

최종 생성된 마크다운 텍스트 토큰을 스트리밍 출력하여 사용자 대기 시간(Time-to-First-Token)을 극소화.

3. 반응형 기상 아키텍처 (Reactive Wakeup Engine)

[핵심 아키텍처 원칙] Busy Polling의 원천 배제와 Reactive Wakeup Zero-Polling Protocol

기존의 나이브한 에이전트는 백그라운드 태스크나 서브에이전트의 종료를 대기하기 위해 while sleep(5): check() 형태의 루프를 돌며 치명적인 토큰 낭비와 컨텍스트 고갈을 유발했다. 안티그래비티는 이를 엄격히 금지하며, **이벤트 드리븐 반응형 기상(Reactive Wakeup)** 메커니즘을 전면 적용한다.

반응형 기상 트리거 소스 (Wakeup Triggers)

- **서브에이전트 통신:** 위임된 서브에이전트가 작업을 마치고 결과를 전송하거나 가이드 요청을 보낼 때 런타임이 부모를 즉시 기상.
- **백그라운드 태스크 완료:** 비동기로 실행 중인 UBT 컴파일러, 패키징 스크립트 종료 시 이벤트 디스패치.
- **사용자 큐 메시지 도착:** 사용자가 에이전트 실행 도중 추가로 전송한 사용자 큐 메시지가 디큐(Dequeue)될 때.
- **타이머 및 크론 만료:** schedule 도구로 등록된 단발성 타이머 만료 또는 크론 주기 도달 시.

기상 동작 시 런타임 상태 복원 흐름

```
# Reactive Wakeup State Transition 1. Agent enters SLEEP state (No tools called) 2. Event Bus receives: "TaskFinished(TaskId=B49, Code=0)" 3. Runtime captures output payload into Context 4. Priority notification injected into Agent turn: "[SYSTEM EVENT] Background task B49 completed successfully." 5. Agent automatically RESUMES execution without user intervention.
```

1. 서브에이전트 계층 아키텍처 (Subagent Hierarchy)

단일 에이전트가 방대한 코드베이스 검색, 대량 컴파일 오류 분석, 다중 파일 리팩토링을 단일 대화 창에서 모두 수행하면 **컨텍스트 윈도우 한계 도달, 어텐션 분산, 속도 저하**가 발생한다. 안티그래비티는 작업을 논리적 단위로 분할하여 독립된 컨텍스트를 가진 서브에이전트에게 위임(Delegate)하는 계층 구조를 채택한다.

self Subagent

부모 에이전트의 전체 도구(파일 편집, 터미널 실행 등), 시스템 프롬프트 및 모델 구성을 그대로 상속.

용도: 독립적인 작업 분기가 필요하지만 부모와 동일한 완전한 권한이 요구되는 복잡한 구현 작업.

research Subagent

읽기 전용 도구(read_file, search_web)만 보유하여 파일이나 시스템을 변이시키지 않음.

용도: 수십 개의 헤더/CPP 파일 탐색, 웹 API 규격 조사 등 부모의 컨텍스트를 오염시키지 않는 대규모 정탐.

define_subagent (Dynamic)

런타임에 특화된 역할, 격리된 도구 권한(EnableWriteTools 등), 전용 시스템 프롬프트를 정의하여 스폰.

용도: DB 스키마 마이그레이션 전문 에이전트, 언리얼 리플렉션 전용 에이전트 등 고도로 특화된 작업.

2. 워크스페이스 격리 모드 (Workspace Isolation Modes)

서브에이전트가 소스코드를 수정할 때 운영 체제 및 Git 레벨에서 부모 워크스페이스와의 충돌을 방지하기 위해 3가지 격리 모드를 제공한다.

격리 모드	작동 원리 (Mechanism)	리소스 및 오버헤드	최적 사용 사례
inherit (기본)	부모와 동일한 실제 디렉터리 경로를 직접 공유하여 파일 작업 수행.	오버헤드 제로, 즉각적 파일 변경 반영.	단일 파일의 사소한 편집, 정적 분석 및 컴파일 검증.
branch	부모 저장소로부터 완전히 독립된 격리 워크스페이스 폴더를 물리적으로 복제/분기.	디스크 복제 시간 및 용량 소모 발생.	실패 가능성이 높은 파괴적 대규모 아키텍처 리팩토링, 실험적 빌드.
share	Git worktree 또는 Mercurial hg share 메커니즘을 활용하여 단일 스토리지를 기반으로 독립 브랜칭.	스토리지 복제 없이 거의 0에 가까운 디스크 오버헤드로 독립성 제공.	고용량 언리얼 프로젝트(FC)에서 안전한 멀티 브랜치 동시 수정 및 테스트.

3. 서브에이전트 프로세스 간 통신 (IPC) 및 생명주기 관리

[Parent Agent] ——invoke_subagent(Role="Code Researcher", Type="research")——→ **[Subagent Runtime]**
|
|←———————— Returns unique ConversationId (e.g., "conv-9f82a") ————————————————————|
|
• Parent stops calling tools → enters Sleep (Reactive Wait)
• Subagent independently queries 20+ files, analyzes symbols, synthesizes findings
|
|←———————— Subagent sends completion payload / artifact link ————————————————————|
|
[Parent Resumes] → Ingests synthesized findings → Calls manage_subagents(Action="kill") → Workspace Cleaned.

[통신 프로토콜 주의사항] send_message의 용도 제한 Protocol Strictness

send_message 도구는 시스템 내부에서 부모와 서브에이전트 간의 **프로세스 간 데이터 교환(IPC)** 전용이다. 사용자에게 응답을 전달할 때 이 도구를 사용해서는 절대 안 되며, 사용자와의 소통은 반드시 **최종 가시적 텍스트 출력**을 통해 이루어져야 한다.

1. 비동기 백그라운드 태스크 엔진 (Background Task Engine)

언리얼 엔진 빌드(UBT), 셰이더 컴파일, 패키징 등의 작업은 수 분 이상 소요될 수 있다. 안티그래비티는 동기식 블로킹으로 인한 타임아웃을 방지하기 위해 **자동 비동기 전환 및 데몬 태스크 관리 아키텍처**를 제공한다.

일반 명령어의 비동기 승격 (Async Escalation) Dynamic Backgrounding

run_command 실행 시 WaitMsBeforeAsync(500~10000ms) 시간을 지정한다. 지정된 시간 내에 명령어가 종료되지 않으면, 시스템은 이를 자동으로 **백그라운드 태스크**로 전환하고 고유 TaskId를 발급한 뒤 에이전트에게 제어권을 즉시 반환한다.

상시 데몬 프로세스 (Daemon Processes) Long-Running Support

개발 웹 서버, Live Coding 리스너, 파일 감시자(File Watcher)와 같이 종료되지 않고 무기한 동작해야 하는 프로세스는 IsDaemon: true 플래그를 설정하여 스폰한다. 시스템은 프로세스가 종료되지 않더라도 성공적으로 백그라운드에 안착했음을 확인하고 에이전트를 차단하지 않는다.

2. 백그라운드 태스크 제어 인터페이스 (manage_task)

Action	파라미터	동작 설명 및 내부 메커니즘	반환 데이터
list	-	현재 시스템에서 백그라운드로 실행 중인 모든 태스크의 목록과 상태를 조회.	TaskId, 실행 명령, 실행 시간, 상태
status	TaskId (필수)	지정된 태스크의 최신 표준 출력(stdout) 및 표준 에러(stderr) 버퍼 로그를 실시간 확인.	최근 N줄 로그 버퍼, CPU/상태
send_input	TaskId, Input (필수)	실행 중인 대화형 프로세스의 stdin 파이프로 텍스트 입력을 직접 주입 (예: Y/N 확인, 디버거 명령).	입력 전달 성공 여부
kill	TaskId (필수)	비정상 루프에 빠진 태스크 또는 불필요해진 백그라운드 프로세스 프로세스 트리를 강제 종료 (SIGTERM/SIGKILL).	프로세스 종료 확인 코드

3. 스케줄링 엔진 (schedule Engine)

에이전트가 특정 시점이나 주기마다 작업을 자율 수행하도록 타이머 및 크론(Cron) 작업을 스케줄링할 수 있다. **쉘의 sleep 명령어를 절대 사용하지 않고** 스케줄러를 통해 이벤트를 수신한다.

단발성 지연 타이머 (One-Shot Timer)

```
schedule(DurationSeconds=300, Prompt="300초 경과: 서버 패키징 완료 여부 확인")
```

지정된 초 단위 시간 경과 후 시스템이 높은 우선순위 알림(Prompt)을 발행하여 에이전트를 깨움. TimerCondition을 설정하면 특정 메시지 도착 시 타이머를 조기 취소할 수 있음.

주기적 크론 작업 (Recurring Cron Job)

```
schedule(CronExpression="*/10 * * * *", Prompt="10분 주기: 깃 리포지토리 동기화 점검")
```

표준 5-field 크론 문법(분 시 일 월 요일)을 지원. MaxIterations를 통해 최대 반복 횟수를 제한하거나, IsDaemon: true로 현재 대화 세션 종료 후에도 영속 동작하게 설정 가능.

[아키텍처 통합] 비동기 태스크와 스케줄러의 유기적 결합 흐름 Workflow Automation

대규모 언리얼 프로젝트에서 에이전트는 장시간 컴파일 작업(pipeline.py build)을 비동기 태스크로 구동한 후, 별도의 지연 루프 없이 즉시 턴을 종료한다. 태스크가 종료되면 운영체제 시그널이 에이전트 런타임 이벤트 버스로 전달되어 에이전트를 자동 기상시키며, 에이전트는 컴파일 결과를 읽어 즉각 다음 단계(자율 패치 또는 테스트)를 수행한다.

1. 도구 실행 정책 (Tool Execution Policy) 4단계

안티그래비티는 자율성과 안정성 사이의 균형을 맞추기 위해 사용자가 환경의 신뢰 수준에 따라 선택할 수 있는 **4단계 도구 실행 정책 모델**을 엄격히 적용한다.

정책 모드	도구 실행 시 동작 (Behavior)	적용 대상 환경	위험 방어 수준
always-proceed	사용자 개입 없이 모든 파일 읽기/쓰기, 터미널 명령어, 도구 호출을 전자동 실행.	신뢰된 로컬 프로젝트, 고속 자동화 CI/CD	주의 (신뢰 환경)
request-review (기본 권장)	읽기 작업은 자동 실행하되, 파일 생성/수정, 쉘 명령어 등 시스템 변이 시 승인 모달 출력.	일반 개발 환경 및 페어 프로그래밍	균형적 (표준)
strict	단순 파일 읽기를 포함한 모든 단일 도구 호출마다 사용자의 명시적 승인을 요구.	극도의 보안이 요구되는 레거시/운영 서버	최고 보안 (수동)
proceed-in-sandbox	모든 터미널 명령어와 프로세스 실행을 격리된 **터미널 샌드박스** 내부로 강제 라우팅.	외부 오픈소스 분석, 비검증 패키지 테스트	완전 격리 (안전)

2. 터미널 샌드박스 및 리소스 접근 제어 아키텍처

1. 파일 시스템 가상화 및 경계 제어 File Boundary

에이전트의 작업 영역은 원칙적으로 워크스페이스 루트(Workspace Root)로 한정된다.

- **Non-Workspace File Policy:** 루트 외부 파일 접근 시 allow, ask, deny 3단계 정책 적용.
- **Path Traversal 차단:** 상대 경로(../../)를 통한 OS 시스템 파일(Windows System32, /etc 등) 접근 원천 차단.

2. 네트워크 및 브라우저 도메인 제어 Network Policy

에이전트의 외부 웹 탐색 및 API 요청에 대한 통신 채널을 제한한다.

- **Internet Access Policy:** 전역 허용(allow), 질의(ask), 완전 차단(deny).
- **Browser Domain Allowlist:** 브라우징 도구가 접속 가능한 호스트를 화이트리스트로 제한하여 정보 유출 방지.

3. 전역 설정(Global)과 프로젝트별(Project-Level) 오버라이드 계층

[Global Settings] (~/.gemini/config/)
• Active Model Selection (Gemini 3.8 Flash/Pro)
• Default Tool Execution Policy: request-review
• Global Command Denylist: rm -rf /, format, reg delete

| [Overrides when project is active]
v

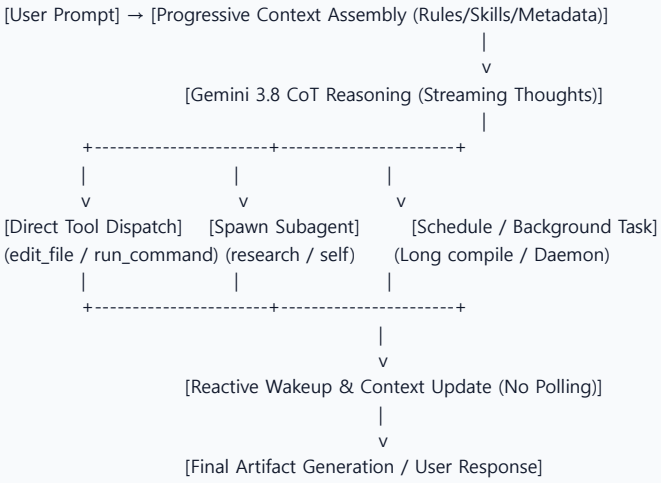
[Project-Level Settings] (Workspace .agents/ or .antigravity/)
• Tool Execution Policy: always-proceed (신뢰된 FC 언리얼 엔진 워크스페이스 내에서 파이프라인 자동화 허용)
• Sandbox Mode: Disabled for local UBT build acceleration
• Non-Workspace File Access: ask (엔진 설치 폴더 Engine/Source 읽기 허용)

[보안 거버넌스 원칙] Command Denylist와 최소 권한 원칙 Least Privilege

프로젝트 설정에서 always-proceed를 채택하더라도, 전역 설정의 Command Denylist(치명적 삭제 명령, 디스크 포맷, 레지스트리 변이 등)는 프로젝트 레벨에서 결코 오버라이드할 수 없도록 설계되어 개발자의 로컬 머신 안정성을 무조건 보장한다.

1. 안티그래비티 런타임 엔드투엔드(End-to-End) 생명주기 종합

개발자의 프롬프트 입력부터 결과물 생성 및 세션 종료에 이르는 런타임 파이프라인의 전체 상태 전이 흐름도이다.



2. 개발자를 위한 런타임 핵심 운영 수칙 5선 (Operational Checklist)

1. Reactive Wakeup 원칙 엄수
명령어나 서브에이전트의 종료를 확인하기 위해 반복적인 sleep이나 상태 체크 루프를 절대 작성하지 않는다. 작업을 비동기로 시작하고 즉시 턴을 종료하면 시스템이 알아서 기상시킨다.

2. Research 서브에이전트 적극 활용
방대한 레포지토리 전수 조사나 외부 기술 문서 탐색 시 부모 에이전트의 메인 대화창에 모든 파일 내용을 적재하지 말고, 읽기 전용 research 에이전트에게 조사를 위임하여 요약본만 수취한다.

3. 파괴적 실험 시 Branch 모드 지정
코드베이스 전체에 영향을 미치는 실험적 리팩토링이나 빌드 테스트는 Workspace="branch" 또는 "share" 모드로 서브에이전트를 구동하여 메인 작업 트리의 오염을 원천 방지한다.

4. 장기 태스크의 명시적 데몬화
종료되지 않는 라이브 서버나 감시 프로세스는 IsDaemon: true를 설정하여 스폰함으로써 에이전트가 무의미하게 타임아웃을 기다리다 실패 처리되는 현상을 방지한다.

3. 제1권 결론 및 제2권 연계 안내

본 제1권에서는 안티그래비티 플랫폼의 구조적 토대인 4대 서피스, 비동기 추론 루프, 서브에이전트 IPC, 스케줄링 및 보안 거버넌스를 살펴보았다. 이어지는 **[제2권: 커스터마이징 아키텍처 & 확장성 프레임워크]**에서는 에이전트를 도메인 전문가로 진화시키는 5대 기둥(Rules, Skills, Plugins, Hooks, MCP)과 점진적 공개(Progressive Disclosure), 아티팩트 시스템 및 트랜스크립트 상태 보존 메커니즘을 상세히 분석한다.

[Next Volume] 제2권 핵심 예고 Volume 2 Preview

Rules & Skills 아키텍처, hooks.json 이벤트 가로채기, mcp_config.json을 통한 언리얼 엔진 5.8 실시간 연동 프로토콜, Generative UI 및 Markdown 아티팩트 엔진 설계 표준 수록.